

ANÁLISIS DEL RETO

Alejandro Zuluaga, 202525929, a.zuluagar@uniandes.edu.co

Jerónimo Gómez, 202420510, j.gomezs2@uniandes.edu.co

Requerimiento 1

Descripción

Para este requerimiento queremos filtrar los equipos por marca y factor de forma.

Primero se seleccionan los que cumplen los filtros, se calcula el precio promedio, se ordenan los filtrados de mayor a menor precio (si son iguales se desempata por el modelo) y se retorna el tiempo de ejecución, total que cumple con los filtros, precio promedio y la lista de computadores (si son más de 20, los primeros y últimos 10 de la lista)

Entrada	catalogo, brand, form_factor
Salidas	{“excecution_time”:float, “total_count”:int, “average_price”:float, “computers”:list}
Implementado (Sí/No)	Sí, implementado por Alejandro Zuluaga

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Acceso al hashmap	$O(1)$
Filtrado por marca	$O(n)$
Cálculo promedio	$O(m)$ (m =computadores filtrados)
Ordenamiento (mergesort)	$O(M \log M)$
Seleccionar primeros/ultimos 10	$O(1)$
TOTAL	$O(N \log N)$ peor caso.

Análisis

El cuello de botella está en el ordenamiento con mergesort, que garantiza $O(M \log M)$ en todos los casos (mejor que quicksort en el peor caso). El filtrado inicial tiene complejidad $O(n)$ porque aunque podemos filtrar por factor de forma en $O(1)$ (hashmap), luego debemos filtrar por marca recorriendo todos los elementos de dicho factor de forma. El cálculo del promedio debe recorrer todos los computadores

filtrados por lo que también es $O(n)$ en el peor caso. $n = \#$ de computadores con el factor de forma especificado.

Requerimiento 3

Descripción

Para este requerimiento queremos filtrar los equipos por marca y factor de forma.

Primero se seleccionan los que cumplen los filtros, se calcula el precio promedio, se ordenan los filtrados de mayor a menor precio (si son iguales se desempata por el modelo) y finalmente se retornan los primeros n .

Entrada	catalog, gpu_model, brand, n
Salidas	{“excecution_time”:float, “total_count”:int, “average_price”:float, “computers”:list}
Implementado (Sí/No)	Sí, implementado por Jerónimo Gómez

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Acceso al hashmap por gpu_model	$O(1)$
Filtrado por brand	$O(k)$ donde k es el $\#$ de equipos con el gpu_model elegido
Ordenamiento (Merge Sort)	$O(m \log m)$ $m = \#$ que cumplen el filtro
Seleccionar primeros n	$O(n)$
TOTAL	$O(k+m^2)$

Análisis

El hash map permite acceder a los equipos que cuentan con el gpu_model seleccionado con tan solo $O(1)$, así se evita recorrer todos los datos ($O(n)$). El cuello de botella aquí sería el merge sort en los equipos m filtrados $O(m \log m)$. Sin embargo, en la práctica m es mucho menor a n haciendo que el rendimiento de la búsqueda y el ordenamiento sea mucho mejor que una búsqueda sobre todo el dataset.

Requerimiento 4

Descripción

Queremos obtener el precio, vram, ram, y cpu_boost promedio de computadores que cumplen con una combinación de CPU Brand - GPU model, así como los 2 más costosos

Primero, seleccionamos los que cumplen con el filtro GPU model (hashmap), y hacemos un recorrido para filtrar por CPU brand. Después calculamos los promedios de precio, vram, ram y cpu_boost de los que cumplen los filtros. Con Merge Sort, ordenamos los filtrados por precio (caso de empate por peso).

Entrada	catalogo, cpu brand, gpu model
Salidas	{“excecution_time”:float, “total_count”:int, “average_price”:float, “average_vram”:float, “average_ram”:float, “average_cpu_boost”:float, “top_2”:dict}
Implementado (Sí/No)	Sí, implementado en equipo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Acceso al hashmap	$O(1)$
Filtrado por cpu brand	$O(n)$
Cálculo promedio	$O(m)$ (m=computadores filtrados)
Ordenamiento (mergesort)	$O(M \log M)$
Seleccionar primeros/ultimos 10	$O(1)$
TOTAL	$O(N \log N)$ peor caso.

Análisis

El cuello de botella está en el ordenamiento con mergesort, que garantiza $O(M \log M)$ en todos los casos (mejor que quicksort en el peor caso). Aunque este no es necesario pues se podrían encontrar los 2 mayores en $O(n)$, sí hace que sea extensible si quisiéramos encontrar los k mayores.

Requerimiento 5

Descripción

Queremos obtener los N equipos mejor equipados según factor de forma y marca en un rango de años. Primero seleccionamos el factor de forma del hashmap, luego recorremos para filtrar por brand y año, e incrementamos los contadores de intel o amd para los que cumplen el filtro. Después, usando mergesort, ordenamos los computadores filtrados por mayor RAM, o desempataamos por cpu_boost o por precio si también es igual. Finalmente seleccionamos los primeros n elementos de la lista (o toda si $size < n$)

Entrada	catalogo, n, año inicial, año final, brand, factor de forma.
Salidas	{“excecution_time”:float, “total_count”:int, “intel_count”:int, “amd_count”:int, “top_n”:list}
Implementado (Sí/No)	Sí, implementado en equipo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Acceso al hashmap	$O(1)$
Filtrado por año y brand	$O(N)$
Contar intel vs amd	$O(M)$ (M=computadores filtrados)
Ordenamiento (mergesort)	$O(M \log M)$
Seleccionar primeros k	$O(k)$
TOTAL	$O(N \log N)$ peor caso.

Análisis

El cuello de botella está en el ordenamiento con mergesort, que garantiza $O(M \log M)$ en todos los casos (mejor que quicksort en el peor caso). De resto todas las operaciones son recorridos sencillos con complejidad lineal $O(N)$.

Requerimiento 6

Descripción

Queremos obtener los n laptops mejor equipados según factor de forma y tipo de pantalla y su valor de eficiencia

Primero, seleccionamos los que cumplen con el filtro form factor (hashmap), y hacemos un recorrido para filtrar los laptops con ese tipo de pantalla, además, por cada filtrado calculamos el valor de eficiencia ($\text{battery_wh} * \text{cpu_boost} / \text{charger_watts}$) y sumamos a los contadores de windows o linux. Con Merge Sort, ordenamos los filtrados por valor de eficiencia (desempate por precio). Seleccionamos y retornamos los primeros n.

Entrada	catalogo, factor de forma, display type, n
Salidas	result:arraylist, total:int, windows_count:int, linux_count:int
Implementado (Sí/No)	Sí, implementado en equipo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Acceso al hashmap, filtro por factor de forma	$O(1)$
Filtrado por laptops con tipo de pantalla, calculo valor de eficiencia y suma a contador	$O(K)$ K= total con factor de forma
Ordenamiento (mergesort)	$O(M \log M)$ M= total que cumplen filtro
Seleccionar primeros/ultimos n	$O(n)$
TOTAL	$O(N \log N)$ peor caso. N=tamaño catalogo

Análisis

El cuello de botella está en el ordenamiento con mergesort, que garantiza $O(M \log M)$ en todos los casos (mejor que quicksort en el peor caso). Tanto el cálculo del valor de eficiencia como la suma de los contadores $O(1)$ se hacen en un solo recorrido $O(K)$